

JAVA LAB WEEK 1&2

1. Basic Information

Experiment No.: 01

Date: 17/09/2026

Title: Bank Account Management System using Inheritance

2. Objective / Aim

To design and implement a **Bank Account Management System in Java** using classes, objects, constructors, inheritance, method overriding, and basic banking operations such as deposit, withdrawal, interest calculation, and money transfer.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code
- Command Prompt/Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Create a superclass Account.
3. Declare account number, balance, and account type as attributes.
4. Create a parameterized constructor in Account to initialize account details.
5. Define the details(), deposit(), withdraw(), and transfer() methods in the Account class.
6. Create a subclass SavingsAccount that extends Account.
7. Use the super() constructor to initialize the account number, balance, and account type.

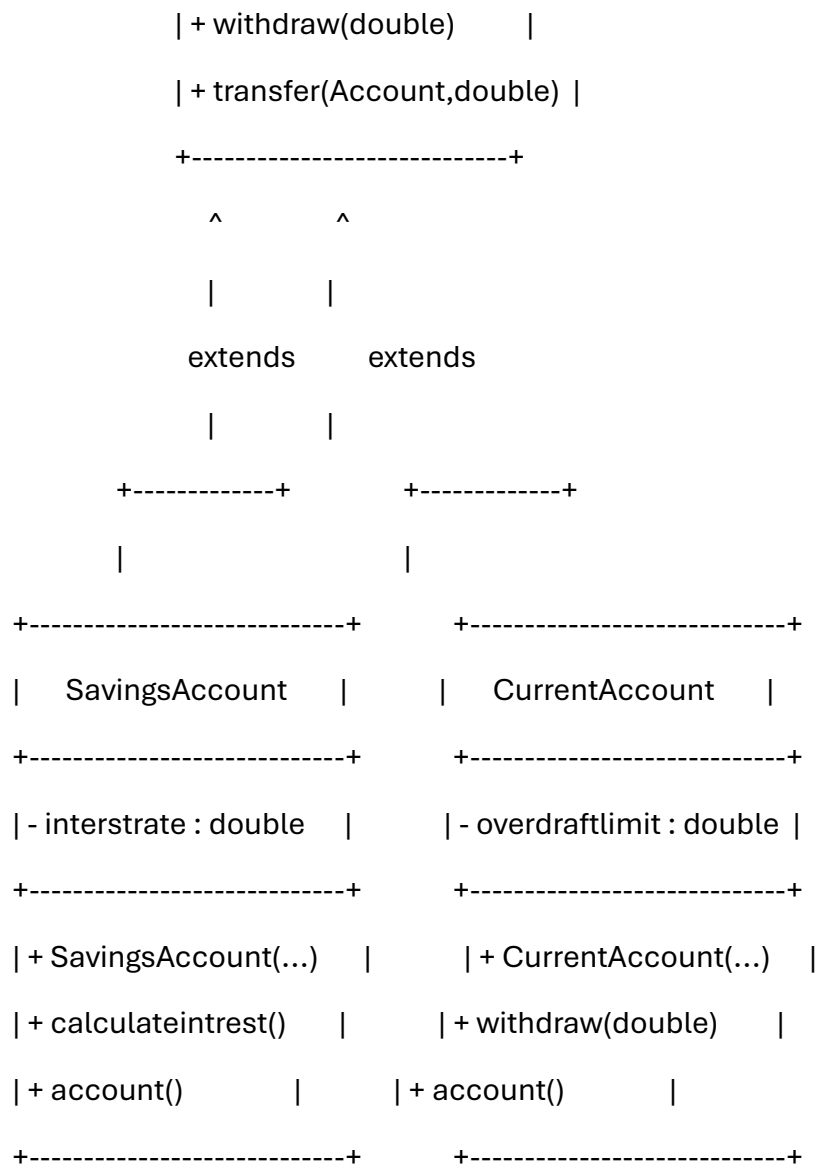
8. Declare the interest rate in SavingsAccount.
9. Define the calculateintrest() method to calculate and add interest to the balance.
10. Define the account() method to display savings account details.
11. Create another subclass CurrentAccount that extends Account.
12. Declare the overdraft limit in CurrentAccount.
13. Override the withdraw() method to allow withdrawal up to the balance plus overdraft limit.
14. Define the account() method to display current account details.
15. Create objects s and c for SavingsAccount and CurrentAccount.
16. Display the initial details of both accounts.
17. Deposit money into the savings account.
18. Withdraw money from the current account.
19. Calculate interest for the savings account.
20. Transfer money from the savings account to the current account.
21. Withdraw money from the current account.
22. Display the final account details.
23. Stop the program.

5. UML Class Diagram

```

+-----+
|   Account   |
+-----+
| - accnum : int      |
| - balance : double  |
| - acctype : String  |
+-----+
| + Account(int,double,String)|
| + details()          |
| + deposit(double)    |

```



6. Source Code

```

class Account{
    int accnum;

    double balance;

    String acctype;

    public Account(int n,double m,String l){

        this.accnum=n;

        this.balance=m;

        this.acctype=l;

    }
  
```

```
void details() {  
    System.out.println("accnum :"+accnum);  
    System.out.println("balance :"+balance);  
    System.out.println("acctype :"+acctype);  
}  
void deposit(double amount) {  
  
    balance = balance + amount;  
    System.out.println("Deposited: " + amount);  
  
}  
void withdraw(double amount){  
    if (amount<=balance) {  
        balance = balance - amount;  
        System.out.println("Withdrawn: " + amount);  
    } else {  
        System.out.println("Insufficient balance");  
    }  
}  
void transfer(Account receiver,double amount) {  
    if(amount > 0 && amount <= balance) {  
        this.balance = this.balance - amount;  
        receiver.balance = receiver.balance + amount;  
        System.out.println("Transferred: " + amount);  
        System.out.println("To Account: " + receiver.accnum);  
    }else {  
        System.out.println("Transfer failed");  
    }  
}
```

```

    }
}

class SavingsAccount extends Account{

    double interstrate;

    public SavingsAccount(int n,double m,double k){

        super(n,m,"saving account");

        this.interstrate=k;

    }

    void calculateintrest() {

        double intrest = balance * (interstrate / 100);

        balance = balance + intrest;

        System.out.println("value :"+intrest);

    }

    void account() {

        System.out.println("accnum :"+accnum);

        System.out.println("balance :"+balance);

        System.out.println("interstrate :"+interstrate);

    }

}

class CurrentAccount extends Account{

    double overdraftlimit;

    public CurrentAccount(int n,double m,double o){

        super(n,m,"current account");

        this.overdraftlimit=o;

    }

    @Override

    void withdraw(double amount){

        if (amount > 0 && amount <= balance+overdraftlimit) {

```

```

        balance = balance - amount;

        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("exceed overdraft limit");
    }
}

void account() {
    System.out.println("accnum :"+accnum);
    System.out.println("balance :"+balance);
    System.out.println("overdraftlimit :"+overdraftlimit);
}
}

public class Bank {

    public static void main(String []args) {
        SavingsAccount s = new SavingsAccount(3000,2424,50);
        CurrentAccount c = new CurrentAccount(2254,5845,300);
        System.out.println("savings account");
        s.account();
        System.out.println("\ncurrent account");
        c.account();
        System.out.println("\nDeposit");
        s.deposit(1000);
        System.out.println("\nwithdraw");
        c.withdraw(2000);
        System.out.println("\ninterstrate");
        s.calculateintrest();
        System.out.println("\ntransfer");
        s.transfer(c,500);
    }
}

```

```
        System.out.println("\nwithdraw");
        c.withdraw(5000);
        System.out.println("\nfinal account details\n");
        s.account();
        c.account();
    }
}
```

7. Compilation & Execution Commands

Java Filename: Bank.java

Compilation

javac Bank.java

Execution

java Bank

8. Output Screen

savings account

accnum :3000

balance :2424.0

interstrate :50.0

current account

accnum :2254

balance :5845.0

overdraftlimit :300.0

Deposit

Deposited: 1000.0

withdraw

Withdrawn: 2000.0

interstrate

value :1712.0

transfer

Transferred: 500.0

To Account: 2254

withdraw

Withdrawn: 5000.0

final account details

accnum :3000

balance :4636.0

interstrate :50.0

accnum :2254

balance :-1155.0

overdraftlimit :300.0

9. Result

The **Bank Account Management System** was successfully implemented and executed in Java. The program demonstrates **classes, objects, constructors, inheritance, method overriding, super() keyword, encapsulation of account operations, deposit, withdrawal, interest calculation, money transfer, and overdraft handling.**